

Date: / /

Sat Sun Mon Tue Thu Wed Fri

Subject: -----

Fun $f(a, b, c) = \text{if } c(a) \text{ then append}(a, b)$ (الف 1)
 else append($a, f(a-1, b, c)$)

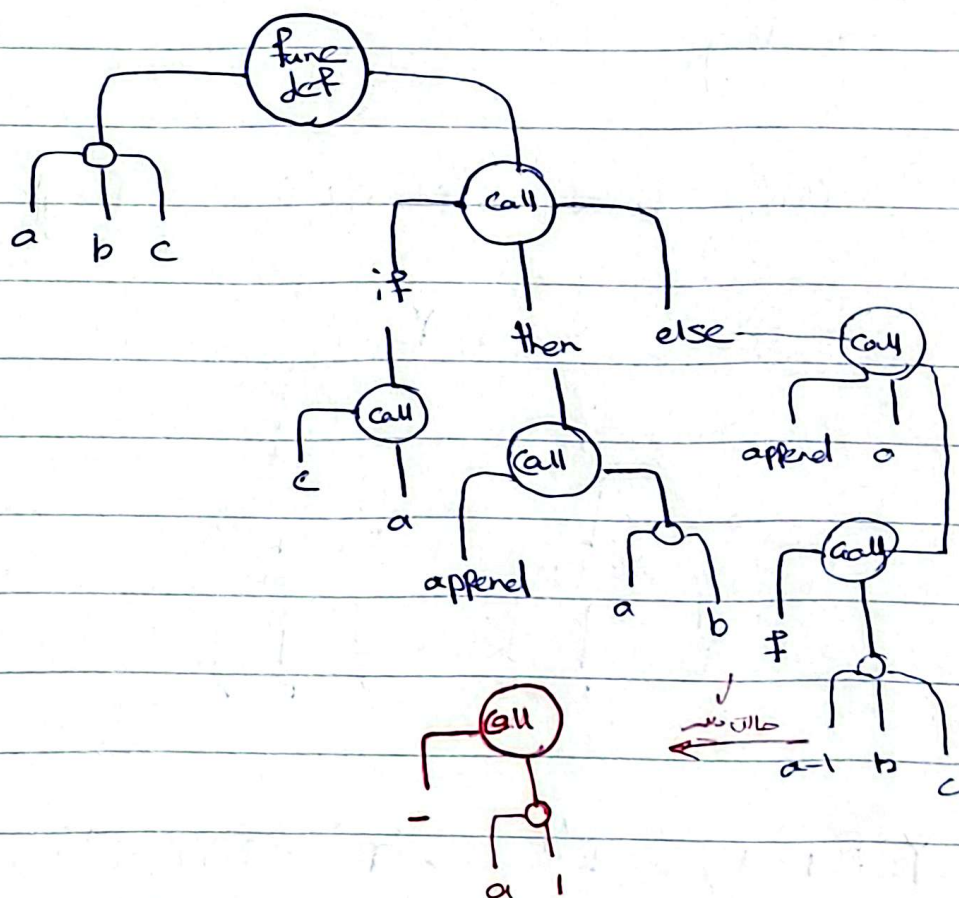
$a-1 \rightarrow a: \text{int}$

$c(a): \text{boolean} \rightarrow c: \text{int} \rightarrow \text{boolean}$

append(a, b) $\rightarrow b: \text{int list}, \underbrace{\text{append}(a, b): \text{int list}}_{f(a, b)}$

append($a, f(a-1, b, c)$) $\rightarrow f(a-1, b, c): \text{int list}$

$\rightarrow \text{Fun } f: \text{int} \times \text{int list} \times (\text{int} \rightarrow \text{bool}) \rightarrow \text{int list}$



Date: / /

Sat Sun Mon Tue Thu Wed Fri

Subject: -----

fun $f(a, b, c) = \text{if } b(\text{concat}(c, a(\text{true}))) \text{ then } d(c) \text{ else } a(\text{false})$ (ب)

$b(\text{concat}(c, a(\text{true}))) \rightarrow c: \text{string}$
 $\rightarrow a: \text{boolean} \rightarrow \text{string}$
 $\rightarrow b: \text{string} \rightarrow \text{boolean}$

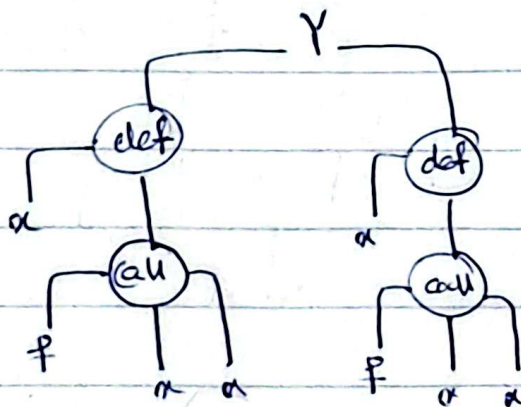
$\text{type}(d(c)) = \text{type}(a(\text{false})) \rightarrow d(c): \text{string}$
 $\rightarrow d: \text{string} \rightarrow \text{string}$
 $\rightarrow \text{type}(f(a, b, c, d)) = \text{string}$

$\rightarrow f: (\text{bool} \rightarrow \text{string}) \times (\text{string} \rightarrow \text{bool}) \times \text{string} \times (\text{string} \rightarrow \text{string})$

$\rightarrow \text{string}$

(ج)

fun $Y(f) = (\text{fn } \alpha \rightarrow f(\alpha \alpha)) (\text{fn } \alpha \rightarrow f(\alpha \alpha))$



بمعنى حاشي
 type
 def
 من سود

$Y: ((T \times T) \rightarrow T) \rightarrow (T \rightarrow ((T \times T) \rightarrow T)) \times \sim \sim$

Date: / /

Sat Sun Mon Tue Wed Fri

Subject: _____

(۲) تابع زیر تابع g می باشد که با شرف راری تابع a چنانچه a فقط عناصری که این شرف را داشته باشند را به دست می دهد.

Fun $f(g, [I]) = [I]$

| $f(g | a :: as) = \text{if } g \ a \text{ then } a :: f(g, as)$
else $f(g, as)$

(۳) عبارات زیر برنامه های n

.method public static fact(I) I

. limit stack

. limit locals 1

. iload_0 [n] (push n)

. ifeq BaseCase (if $n == 0$ go to Base case)

iload_0 [n]

iload_0 [n]

iconst_1 [n, n, 1]

isub [n, n-1]

invokestatic Factorial, fact(I) I [n, fact(n-1)]

imul

ireturn

Base case:

iconst_1 [1]

ireturn

.end method

Date: / /

Sat Sun Mon Tue Wed Fri

Subject: -----

(۴)

خطایا ← باید به جای A و C از LA و CA استفاده کنیم

0	1	2	3
this	i	a	c

این شد محافل مانده را بریزد locals نیاز دارد ←

پس locals limit. استفاده است.

۲- load ← چگونگی ساینر locals را ۲ مقرر کنیم استفاده است.

۳- store ← دوباره به خاطر ساینر استفاده locals خطای دهد

۴- load ← slot دوم شامل A استفاده یک روش است باید از ۲ load استفاده می‌شود.

اعضای g استفاده است. `invokevirtual Alg(LA)I`

۱- load ← باید ۱- load می‌بود چون در اسلات این `int` داریم.

۲- add ← باقی به استفاده خطا، در آخر اسلات یک روش قرار دارد و وقتی تکرار `load` می‌شود.

خروجی تابع `int` است و این در آخر `bytecode`، `return` نوشته شده است.

(۵) مدیریت استفاده شده در gc ما شامل `reference counting`، `marking` و `replacing` است.

(۱) در روش `reference counting` که در زبان پایتون استفاده می‌شود، هر شیء یا متغیر از تعداد شیءهایی که به آن اشاره دارند به می‌داریم و با ساختن `delete` کردن این متغیر را به صفر می‌رسانیم. شیءهایی که `reference count` صفر دارند `garbage` می‌باشند یا پاک می‌شوند.

② در روش marking، یک مجموعه‌ای از object ها (global) دارد به عنوان root در نظر می‌گیریم و به کمک پویشگرها می‌توانیم در واقع هر شیئی که به آن reference ای وجود دارد mark می‌شود و هر شیئی که علامت نخورد باقی مانده در memory باقی می‌ماند. در این روش memory دچار fragmentation می‌شود و ممکن است با وجود داشتن فضای خالی نتوانیم شیئی بزرگ را new کنیم به شکل: mark and sweep

③ replacing and compacting: مانند روش live object دارد به این معنی که آن‌ها به فضای

مجری انتقال می‌دهیم و دیگر fragmentation نخواهیم داشت. در این روش نه برای مثال در memory استفاده شده است، utilization فضای حافظه کاهش می‌یابد

④ با marking cherry از root marking BFS را اجرا می‌کنیم. در هر مرحله از این marking root را به to-space به همراه to-space را خالی می‌کنیم (تعمیم) می‌کنیم. یک forwarding pointer از همان to-space به from-space و پویشگرهای free و scan را اجرا می‌کنیم. زمانی که تمام فرزندان یک node در to-space BFS به to-space است باقی‌مانده آن را به to-space و با میانگین هم to-space و from جای to-space را عوض می‌کنیم.

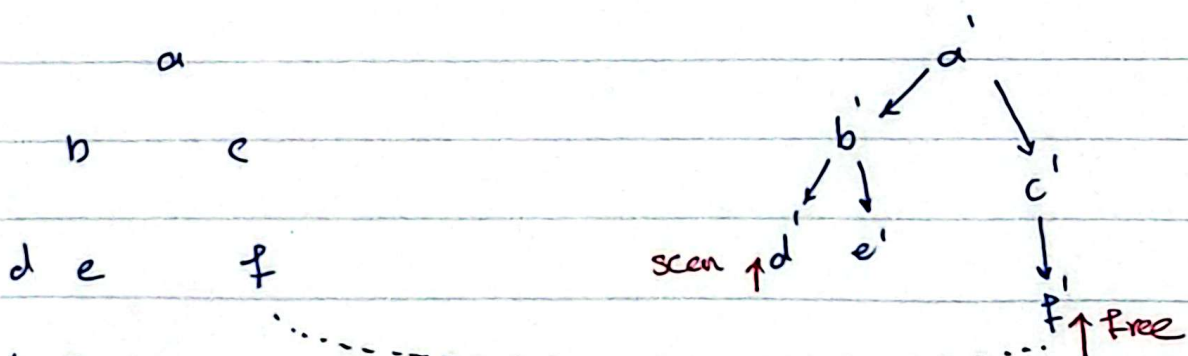
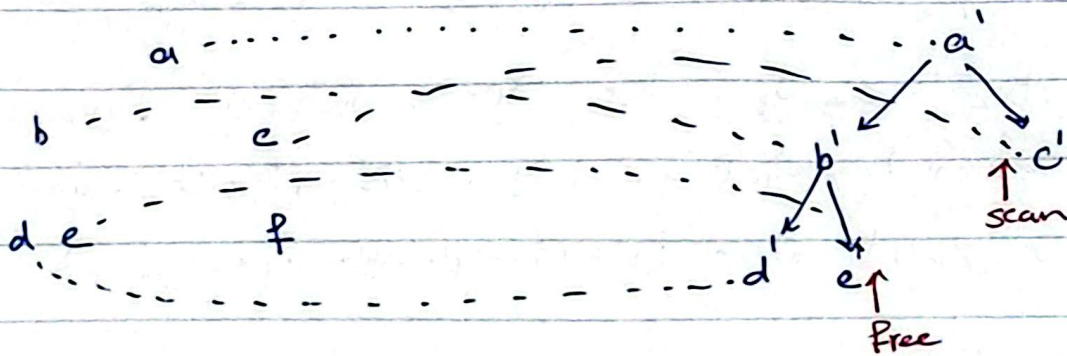
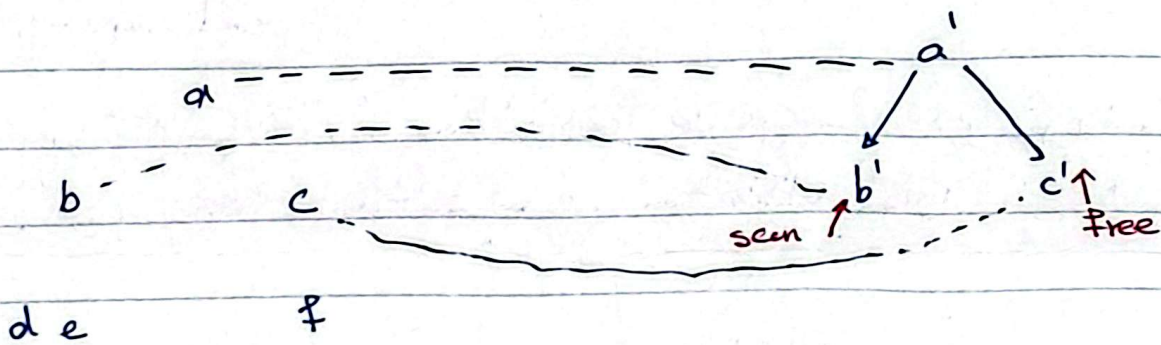
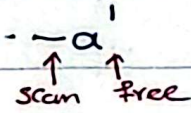
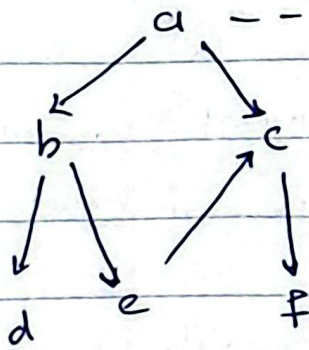
Date: / /

Sat Sun Mon Tue Thu Wed Fri

Subject: -----

from space

to space



نہ از میں صرف scan نہ free ہے ، $e' \neq e'$ وصل ہے مگر $\sum$ و $\frac{1}{e'}$ from
؟

Subject :

Year :

Month :

Date :

۴) برای مقایسه این دو الگوریتم ۳ حالت در نظر می‌گیریم

۱. زمانی که garbage جمع است: الگوریتم mark and sweep در صورتی که heap memory را امتلین می‌کند و garbage را پاک می‌کند. $O(H)$ است.
الگوریتم stop and sweep هم تمام heap را پاک می‌کند و دوباره rate بررسی می‌کند. $O(H)$ می‌شود چون heap utilization آن ۵۰٪ است و مساحتی که پاک می‌شود $\frac{1}{2}$ heap است.

۲. زمانی که garbage جمع نیست: الگوریتم mark and sweep در صورتی که garbage را پاک می‌کند و دوباره rate بررسی می‌کند. $O(H)$ می‌شود چون heap utilization آن ۵۰٪ است و مساحتی که پاک می‌شود $\frac{1}{2}$ heap است.

تعداد لاوهای کمتر است.

۳. زمانی که garbage جمع است: الگوریتم mark and sweep در صورتی که garbage را پاک می‌کند و دوباره rate بررسی می‌کند. $O(H)$ می‌شود چون heap utilization آن ۵۰٪ است و مساحتی که پاک می‌شود $\frac{1}{2}$ heap است.

ج) به این cyclic reference ها می‌گویند که در linked list ها می‌تواند به این شکل باشد: $reference \rightarrow node \rightarrow reference \rightarrow node \rightarrow \dots$ و این باعث می‌شود که garbage جمع نشود و الگوریتم mark and sweep نتواند آن را پاک کند.

د) زمانی که garbage جمع است: الگوریتم mark and sweep در صورتی که garbage را پاک می‌کند و دوباره rate بررسی می‌کند. $O(H)$ می‌شود چون heap utilization آن ۵۰٪ است و مساحتی که پاک می‌شود $\frac{1}{2}$ heap است.

راه پیشنهادی: می‌توانیم به کمک DFS روی این حلقه‌ها عمل کنیم و به این شکل garbage جمع کنیم. $O(H)$ می‌شود چون heap utilization آن ۵۰٪ است و مساحتی که پاک می‌شود $\frac{1}{2}$ heap است.

البته به صورتی که در الگوریتم mark and sweep استفاده می‌کنیم و به این شکل garbage جمع می‌کنیم. $O(H)$ می‌شود چون heap utilization آن ۵۰٪ است و مساحتی که پاک می‌شود $\frac{1}{2}$ heap است.